Ecole Supérieure d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Cyber Sécurité — TP8 — Transport Layer Security / Secure Socket Layer
3e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants :	HATHOUTI Mohammed Taha JIDAL Ilyas KABORE Mohammad Sharif Jonathan
Filière :	Cybersécurité
Année :	2025/2026
Enseignant :	M.SEBBAR & Mme.GADI
Date :	22 avril 2026

Table des matières

Objectifs **3**

1 Installation et prise en main de Wireshark **4**
1.1 Installation 4
1.2 Capture du trafic TCP 4

2 Propriétés de sécurité **5**
2.1 Tableau des propriétés 5
2.2 Vrai/Faux — Questions de sécurité 5
2.2.1 Si le compte email d'Alice `alice@gmail.com` n'est pas piraté, peut-on être sûr que seule Alice peut lire les emails qui lui sont destinés ? 5
2.2.2 Alice utilise SSL pour accéder au site de sa banque. Un adversaire Dolev-Yao ne peut pas voir le mot de passe, mais peut-il savoir qu'Alice accède au site de sa banque ? 5
2.2.3 Si Alice et Bob partagent une clé symétrique secrète k et qu'Alice reçoit un message chiffré avec k, peut-elle être sûre que le message vient de Bob ? 6
2.2.4 Un étudiant réimplémente lui-même les algorithmes cryptographiques à partir d'un livre. Si l'implémentation est correcte, les algorithmes seront-ils sécurisés ? 6
2.2.5 Les sites de phishing ont-ils forcément un certificat invalide ? 6

3 Phishing **6**
3.1 Identification des sites de phishing 6
3.1.1 Site a) : 6
3.1.2 Site b) : 6
3.1.3 Site c) : 7

4 CA vs PGP **7**
4.1 Certificats à vérifier pour accéder à `https://secure_banque.fr` 7
4.2 Un CA peut-il faire une attaque MitM sur la session SSL d'Alice et Bob ? . 7
4.3 Un MitM est-il possible si Alice et Bob utilisent PGP correctement ? . . . 8
4.4 PGP ou SSL : lequel est le plus utilisé sur Internet ? 8

5 Interception de trafic réseau **8**
5.1 Alice accède à une page HTTP 8
5.1.1 Mallory peut-elle voir le contenu de la page ? 8
5.1.2 Mallory peut-elle modifier le contenu ? 8
5.1.3 Mallory peut-elle connaître le nom de la page ? 8
5.2 Alice accède à une page HTTPS 9
5.2.1 Mallory peut-elle voir le contenu de la page ? 9
5.2.2 Mallory peut-elle modifier le contenu ? 9
5.2.3 Mallory peut-elle connaître le nom de la page visitée ? 9

6 Analyse du handshake TLS avec Wireshark **9**
6.1 Rappel : format des cipher suites 9
6.2 Liste des cipher suites disponibles 9
6.3 Analyse de la capture `TLS_Decryption_Anass_Sebbar.pcapng` 10
6.3.1 a) Analyse du champ Client Hello 10
6.3.2 b) Analyse du champ Server Hello 11
6.3.3 c) Observations 12
6.3.4 d) Le serveur a-t-il envoyé un certificat ? 12
6.3.5 e) Le certificat est-il valide ? 12
6.3.6 f) Description de l'échange client-serveur 12
6.3.7 g) Nom de domaine visité 13
6.3.8 h) Le domaine visité utilise-t-il HTTPS ? 13
6.4 Analyse avant/après décryptage 13

Conclusion **15**

Objectifs

L'objectif de ce TP est d'utiliser Wireshark pour capturer et analyser le trafic réseau, avec un focus particulier sur le protocole TLS/SSL. Les points abordés sont les suivants :

- Introduction à Wireshark et analyse du trafic TCP ;
- Identification d'attaques de phishing ;
- Analyse du handshake TLS avec Wireshark ;
- Décryptage de captures TLS à l'aide d'un fichier de clés de session.

1 Installation et prise en main de Wireshark

1.1 Installation

Wireshark a été installé sur la machine Ubuntu avec la commande suivante :

Listing 1 – Installation de Wireshark

```
sudo apt-get install wireshark
sudo wireshark &
```

Au premier lancement, Wireshark affiche une fenêtre vierge. Pour lister les interfaces réseau disponibles, on accède au menu **Capture** → **Interfaces**.

1.2 Capture du trafic TCP

Une capture a été lancée sur l'interface réseau active. Un filtre d'affichage a été appliqué pour isoler uniquement les paquets portant le flag `SYN` :

Listing 2 – Filtre Wireshark pour les paquets SYN

```
tcp.flags.syn == 1
```

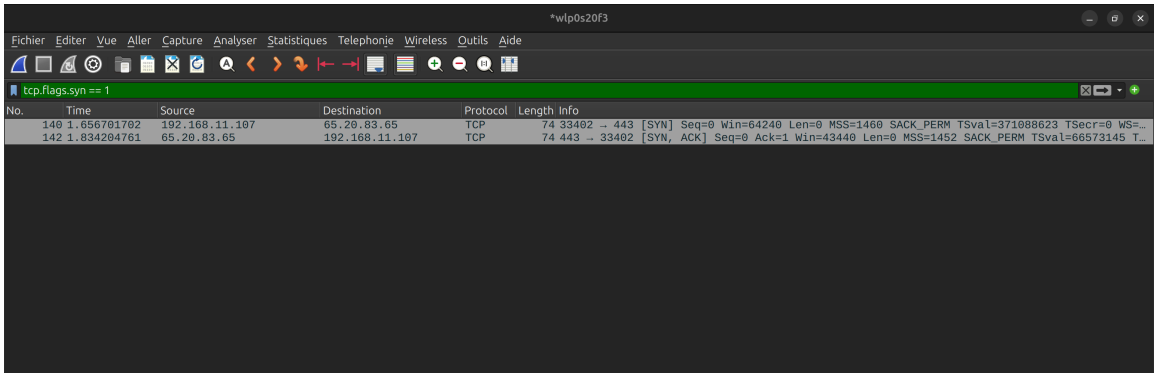

FIGURE 1 – Filtre `tcp.flags.syn == 1` — Paquets SYN et SYN-ACK identifiés

Analyse de la capture :
La capture révèle deux paquets correspondant au début d'un handshake TCP 3-voies :

N°	Source	Destination	Flags	Description
140	192.168.11.107	65.20.83.65	[SYN]	Client initie la connexion (port 33402 → 443)
142	65.20.83.65	192.168.11.107	[SYN, ACK]	Serveur accepte (port 443 → 33402)

TABLE 1 – Paquets SYN capturés — Début du handshake TCP

Observations :

— Le client `192.168.11.107` contacte le serveur `65.20.83.65` sur le port **443** (HTTPS/TLS) ;

— Le troisième paquet du handshake (ACK pur) n'est pas visible car le filtre `tcp.flags.syn == 1` n'affiche que les paquets où SYN=1 ;
— Le fichier de capture contient **587 paquets** au total, dont seulement 2 affichés après filtrage (0,3%).

Pour visualiser le handshake TCP complet, on peut effectuer un clic droit sur le paquet 140 et sélectionner **Suivre** → **Flux TCP**, ce qui affiche l'intégralité de la session.

2 Propriétés de sécurité

2.1 Tableau des propriétés

Algorithme	Confidentialité	Authenticité	Non-répudiation	Disponibilité
MAC	✓	✓	×	×
Signature numérique	×	✓	✓	×

Table 2 – Propriétés de sécurité assurées par MAC et Signature numérique

Justification :

— **MAC** (*Message Authentication Code*) : utilise une clé secrète partagée entre émetteur et récepteur. Il assure la **confidentialité** (seuls les détenteurs de la clé peuvent vérifier) et l'**authenticité** (le message provient bien d'un détenteur de la clé). En revanche, il n'assure pas la **non-répudiation** car n'importe quel détenteur de la clé aurait pu générer le MAC ;
— **Signature numérique** : repose sur la cryptographie asymétrique. La signature avec la clé privée assure l'**authenticité** et la **non-répudiation** (seul le propriétaire de la clé privée peut signer). Elle n'assure pas directement la **confidentialité** (le contenu reste lisible sans déchiffrement).

2.2 Vrai/Faux — Questions de sécurité

2.2.1 Si le compte email d'Alice `alice@gmail.com` n'est pas piraté, peut-on être sûr que seule Alice peut lire les emails qui lui sont destinés ?

Faux. Le fait que le compte ne soit pas piraté ne garantit pas la confidentialité de bout en bout. Les emails transitent en clair sur les serveurs de Google sauf si un chiffrement de bout en bout (PGP, S/MIME) est utilisé. Le fournisseur lui-même (Google) peut lire les messages. De plus, des intermédiaires réseau peuvent intercepter le trafic si TLS n'est pas utilisé pour chaque segment de la chaîne de transmission.

2.2.2 Alice utilise SSL pour accéder au site de sa banque. Un adversaire Dolev-Yao ne peut pas voir le mot de passe, mais peut-il savoir qu'Alice accède au site de sa banque ?

Vrai. SSL/TLS chiffre le contenu de la communication (mot de passe, données échangées), mais le nom de domaine du serveur visité est visible en clair dans le champ **SNI** (*Server Name Indication*) du message *Client Hello* TLS. Un adversaire qui observe le réseau peut donc identifier le site web visité, même s'il ne peut pas lire le contenu.

2.2.3 Si Alice et Bob partagent une clé symétrique secrète k et qu'Alice reçoit un message chiffré avec k, peut-elle être sûre que le message vient de Bob ?

Faux. La possession d'une clé symétrique partagée garantit l'authenticité seulement si la clé est connue exclusivement des deux parties. Cependant, si la clé a été compromise ou si plusieurs parties partagent la même clé, Alice ne peut pas être certaine que le message provient de Bob et non d'un autre détenteur de la clé. De plus, un MAC doit être ajouté pour l'authenticité ; le simple chiffrement symétrique ne l'assure pas.

2.2.4 Un étudiant réimplémente lui-même les algorithmes cryptographiques à partir d'un livre. Si l'implémentation est correcte, les algorithmes seront-ils sécurisés ?

Faux. La sécurité cryptographique ne repose pas uniquement sur la correction algorithmique. Une implémentation correcte peut rester vulnérable aux **attaques par canaux auxiliaires** (*side-channel attacks*) : timing attacks, analyse de consommation électrique, fuites de mémoire, etc. Les bibliothèques cryptographiques professionnelles (OpenSSL, libsodium, openPGP) intègrent des protections contre ces attaques après des années d'audit et de tests. La cryptographie correctement implémentée est extrêmement difficile, et les erreurs d'implémentation sont une source majeure de vulnérabilités réelles.

2.2.5 Les sites de phishing ont-ils forcément un certificat invalide ?

Faux. Un site de phishing peut très bien obtenir un certificat TLS **valide** auprès d'une autorité de certification (par exemple via Let's Encrypt, qui est gratuit et automatisé). La présence du cadenas HTTPS ne garantit que la *confidentialité de la transmission*, pas la *légitimité du site*. Un attaquant peut donc héberger `https://paypal-secure.evil.com` avec un certificat valide pour son domaine, tout en pratiquant le phishing.

3 Phishing

3.1 Identification des sites de phishing

3.1.1 Site a) :

La résolution de la capture d'écran fournie ne permet pas de lire clairement l'URL dans la barre d'adresse. Cependant, on observe que la page affiche une interface imitant le site PayPal. Une analyse plus précise de l'URL serait nécessaire pour conclure définitivement, mais la présentation visuelle suggère une tentative de phishing par usurpation de l'identité PayPal.

3.1.2 Site b) :

Oui, c'est un site de phishing. L'URL affichée dans la barre d'adresse est `https://www.polisjohor.gov.my/templates/beez/chta...`, qui est le domaine officiel de la *police de l'État de Johor* (Malaisie), mais qui héberge une fausse page de connexion PayPal. Ce cas illustre un phishing par **compromission d'un site légitime** : un attaquant a injecté une fausse page PayPal sur un serveur gouvernemental compromis. Le certificat peut être valide (il appartient au vrai domaine `polisjohor.gov.my`), mais le contenu est

frauduleux. La victime voit un cadenas HTTPS vert mais est en réalité sur un faux site PayPal.

3.1.3 Site c) :

Non, ce n'est pas un site de phishing. La capture d'écran montre le site `www.cnn.com` avec son interface habituelle : logo CNN, barre de navigation et articles d'actualité. L'URL dans la barre d'adresse correspond bien au domaine légitime `www.cnn.com`. Il n'y a aucune usurpation d'identité ni tentative de collecter des informations personnelles.

4 CA vs PGP

On considère la hiérarchie : Root CA $\rightarrow$ CA_A $\rightarrow$ CA_C, CA_D ; Root CA $\rightarrow$ CA_B $\rightarrow$ CA_E, CA_F ; CA_D signe un certificat pour `https://secure_banque.fr`.

4.1 Certificats à vérifier pour accéder à `https://secure_banque.fr`

Pour visiter `https://secure_banque.fr`, le navigateur d'Alice doit vérifier la **chaîne de certification** complète remontant jusqu'à une ancre de confiance (Root CA). Dans notre hiérarchie, la chaîne est :

$$\text{Root CA} \rightarrow \mathrm{CA}_A \rightarrow \mathrm{CA}_D \rightarrow \texttt{https://secure_banque.fr}$$

Le navigateur doit donc vérifier **trois certificats** :

1. Le certificat du **site** (`secure_banque.fr`), signé par CA_D ;
2. Le certificat de $\mathbf{CA}_D$, signé par CA_A ;
3. Le certificat de $\mathbf{CA}_A$, signé par Root CA.

Le certificat de Root CA est considéré comme implicitement de confiance (il est pré-installé dans le navigateur/système d'exploitation).

4.2 Un CA peut-il faire une attaque MitM sur la session SSL d'Alice et Bob ?

Oui, théoriquement. Si Alice et Bob utilisent SSL/TLS et que leur communication passe par un certificat signé par un CA compromis ou malveillant, ce CA peut émettre un **faux certificat** pour le domaine d'Alice ou de Bob. Un intermédiaire possédant ce faux certificat peut alors monter une attaque **Man-in-the-Middle** :

— Il présente le faux certificat au client qui le valide (la signature du CA est correcte) ;

— Il établit une connexion TLS séparée avec le vrai serveur ;

— Il décrypte, lit (et éventuellement modifie) tout le trafic entre Alice et Bob.

Des mécanismes comme le **Certificate Pinning** ou **Certificate Transparency** (CT Logs) ont été développés précisément pour détecter ce type d'abus par des CAs malveillants.

4.3 Un MitM est-il possible si Alice et Bob utilisent PGP correctement ?

Non, si Alice et Bob ont échangé leurs clés publiques via un canal de confiance (en personne, via une *Web of Trust* vérifiée, ou en comparant les empreintes de clés). PGP repose sur un modèle de confiance décentralisé : il n'y a pas d'autorité centrale susceptible d'émettre un faux certificat. Si Alice possède la vraie clé publique de Bob et vice versa, un attaquant ne peut pas s'interposer car il ne dispose pas des clés privées correspondantes.

Cependant, si l'échange initial de clés publiques a été compromis (l'attaquant a substitué sa propre clé publique), alors un MitM est possible. La sécurité de PGP repose entièrement sur la confiance établie lors de l'échange de clés.

4.4 PGP ou SSL : lequel est le plus utilisé sur Internet ?

SSL/TLS est de loin le protocole le plus utilisé sur Internet. La raison principale est la **transparence pour l'utilisateur** : TLS est intégré de façon transparente dans les navigateurs et les serveurs web, sans aucune action requise de l'utilisateur final. Le modèle PKI (infrastructure à clé publique) avec les CAs pré-installés dans les navigateurs permet une vérification automatique et immédiate.

PGP, en revanche, nécessite une gestion manuelle des clés (génération, distribution, vérification), ce qui le rend bien plus complexe à déployer à grande échelle. Il reste principalement utilisé pour le chiffrement d'e-mails entre utilisateurs avertis, mais n'a jamais connu d'adoption massive dans le grand public.

5 Interception de trafic réseau

On suppose que Mallory a accès au réseau LAN d'Alice.

5.1 Alice accède à une page HTTP

5.1.1 Mallory peut-elle voir le contenu de la page ?

Oui. HTTP est un protocole en **clair**, non chiffré. Sur un réseau LAN partagé, Mallory peut utiliser un outil comme Wireshark pour capturer tous les paquets transitant sur le réseau. Elle peut reconstituer intégralement le contenu des pages HTTP visitées par Alice, incluant les données de formulaires, les cookies de session, et les mots de passe en clair.

5.1.2 Mallory peut-elle modifier le contenu ?

Oui. En combinant une attaque ARP poisoning (pour se positionner en MitM entre Alice et le routeur) avec une modification des paquets HTTP en transit, Mallory peut altérer le contenu des pages reçues par Alice. Elle peut par exemple injecter du code JavaScript malveillant dans une page légitime.

5.1.3 Mallory peut-elle connaître le nom de la page ?

Oui. L'URL complète est visible en clair dans les en-têtes HTTP (`GET /chemin/page.html HTTP/1.1` et `Host: site.com`).

5.2 Alice accède à une page HTTPS

5.2.1 Mallory peut-elle voir le contenu de la page ?

Non (sous réserve d'une implémentation correcte). HTTPS chiffre l'intégralité du contenu échangé grâce à TLS. Mallory ne peut pas déchiffrer les données sans la clé privée du serveur.

5.2.2 Mallory peut-elle modifier le contenu ?

Non (en principe). TLS assure l'intégrité via un MAC sur chaque enregistrement. Toute modification des données chiffrées est détectée lors de la vérification de l'intégrité, et la connexion est interrompue.

Exception : Si Mallory dispose d'un faux certificat signé par un CA de confiance (voir section CA vs PGP), elle peut monter un MitM TLS et modifier le contenu. C'est pourquoi les navigateurs modernes affichent des avertissements pour les certificats suspects.

5.2.3 Mallory peut-elle connaître le nom de la page visitée ?

Partiellement. Le **nom de domaine** est visible en clair dans le champ **SNI** (*Server Name Indication*) du *Client Hello* TLS. Mallory peut donc savoir qu'Alice visite `www.banque.fr`, mais elle ne peut pas connaître le chemin précis ni les paramètres, qui sont tous chiffrés.

Exemple concret issu de notre capture : Le champ SNI du *Client Hello* révèle le domaine `youtube.com` en clair (paquet 11 : `Client Hello (SNI=youtube.com)`), alors que tout le reste est chiffré.

6 Analyse du handshake TLS avec Wireshark

6.1 Rappel : format des cipher suites

TLS 1.2 : `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`

- `ECDHE` : algorithme d'échange de clés (Elliptic Curve Diffie-Hellman Ephemeral)
- `RSA` : algorithme de signature/authentification
- `AES_128_GCM` : algorithme de chiffrement symétrique
- `SHA256` : algorithme d'intégrité (MAC)

TLS 1.3 : `TLS_AES_128_GCM_SHA256`

- En TLS 1.3, l'échange de clés et l'authentification sont séparés du cipher suite : seuls le chiffrement et l'intégrité y figurent.

6.2 Liste des cipher suites disponibles

Listing 3 – Affichage des cipher suites SSL/TLS disponibles

```
openssl ciphers -v
```

La commande produit la liste suivante (extrait) :

Listing 4 – Extrait de la sortie d'openssl ciphers -v

```
TLS_AES_256_GCM_SHA384          TLSv1.3 Kx=any   Au=any    Enc=
   AESGCM(256)  Mac=AEAD
TLS_CHACHA20_POLY1305_SHA256    TLSv1.3 Kx=any   Au=any    Enc=
   CHACHA20(256) Mac=AEAD
TLS_AES_128_GCM_SHA256          TLSv1.3 Kx=any   Au=any    Enc=
   AESGCM(128)  Mac=AEAD
ECDHE-ECDSA-AES256-GCM-SHA384   TLSv1.2 Kx=ECDH Au=ECDSA Enc=
   AESGCM(256) Mac=AEAD
ECDHE-RSA-AES256-GCM-SHA384     TLSv1.2 Kx=ECDH Au=RSA   Enc=
   AESGCM(256) Mac=AEAD
DHE-RSA-AES256-GCM-SHA384       TLSv1.2 Kx=DH   Au=RSA   Enc=
   AESGCM(256) Mac=AEAD
...
```

La liste montre pour chaque cipher suite les champs :

- **Kx** : mécanisme d'échange de clés (ECDH, DH, RSA, `any` pour TLS 1.3) ;
- **Au** : algorithme d'authentification (ECDSA, RSA, `any` pour TLS 1.3) ;
- **Enc** : algorithme de chiffrement symétrique (AESGCM, CHACHA20/POLY1305, AES en mode CBC...) ;
- **Mac** : algorithme d'intégrité (AEAD pour les suites modernes, SHA1/SHA256/SHA384 pour les anciennes).

On note que les suites TLS 1.3 ont `Kx=any` et `Au=any` car l'échange de clés (toujours ECDHE en pratique) et l'authentification sont gérés séparément du cipher suite dans TLS 1.3.

6.3 Analyse de la capture `TLS_Decryption_Anass_Sebbar.pcapng`

La capture contient **200 paquets** au total, couvrant une session TLS vers `youtube.com` (IP : `172.217.6.46`) depuis le client `192.168.0.46`.

6.3.1 a) Analyse du champ Client Hello

Le paquet 11 correspond au *Client Hello*. La figure ci-dessous montre son contenu déplié dans Wireshark :

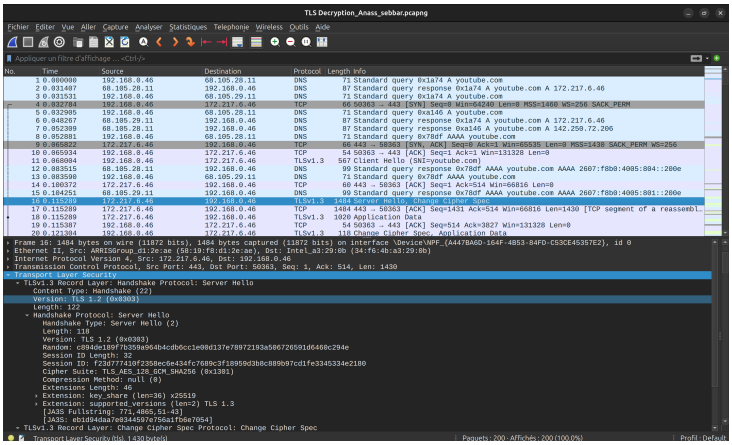

FIGURE 2 – Paquet 11 — Client Hello (SNI=youtube.com) déplié dans Wireshark

Le *Client Hello* contient les informations suivantes :

— **Version de l'enregistrement** : TLS 1.0 (0x0301) — utilisée pour la rétrocompatibilité ;

— **Version proposée dans le Hello** : TLS 1.2 (0x0303), avec support de TLS 1.3 via l'extension `supported_versions` ;

— **Random** : `984b3afd9ce24359381e50c87a133a2e98d07ae510c130c3552d3512ee1bd35f` ;

— **Session ID** : 32 octets ;

— **Cipher Suites** : 18 suites proposées (TLS 1.3 et TLS 1.2) ;

— **Extensions visibles** : `server_name` (SNI = `youtube.com`), `extended_master_secret`, `renegotiation_info` ;

— **SNI** : `youtube.com` — le nom de domaine visité est transmis en clair.

6.3.2 b) Analyse du champ Server Hello

Le paquet 16 correspond au *Server Hello*. La figure ci-dessous montre son contenu :

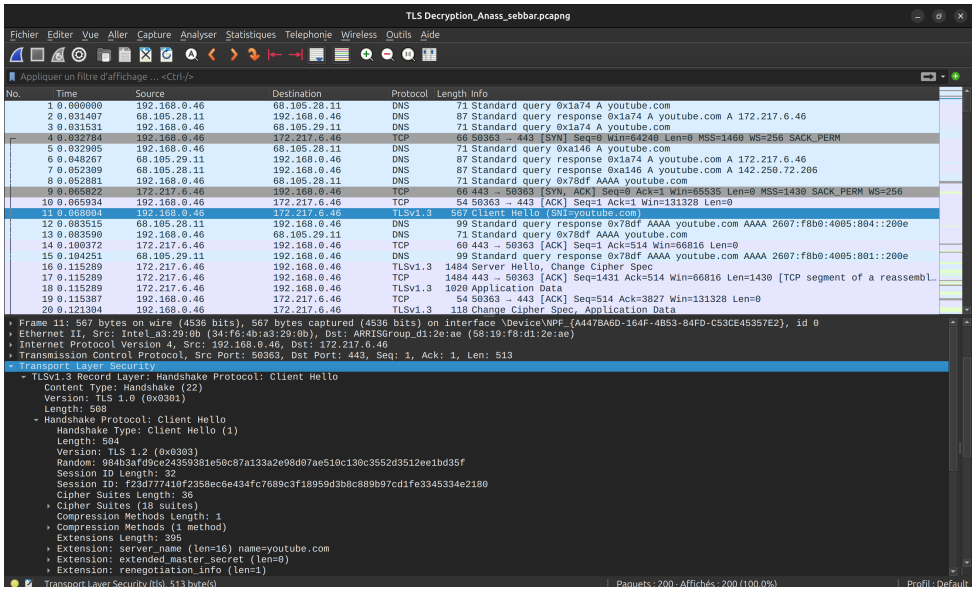

FIGURE 3 – Paquet 16 — Server Hello déplié dans Wireshark

Le *Server Hello* contient les informations suivantes :

— **Version de l'enregistrement** : TLS 1.2 (0x0303) — champ hérité, présent pour compatibilité ;

— **Version négociée réelle** : **TLS 1.3**, confirmée par l'extension `supported_versions` ;

— **Random** : `c894de189f7b359a964b4cdb6cc1e00d137e78972193a506726591d6460c294e` ;

— **Session ID** : même valeur que le client ;

— **Cipher Suite sélectionnée** : `TLS_AES_128_GCM_SHA256` **(0x1301)** ;

— **Extensions** : `key_share` (len=36) pour l'échange ECDHE, `supported_versions` TLS 1.3.

6.3.3 c) Observations

On observe plusieurs éléments importants dans cet échange :

— **Négociation de TLS 1.3** : bien que les en-têtes affichent TLS 1.2 pour compatibilité (*middlebox compatibility mode*), la version réellement négociée est TLS 1.3 ;

— **Cipher suite légère** : `TLS_AES_128_GCM_SHA256` utilise AES-128 en mode GCM (chiffrement authentifié) avec SHA-256 pour la dérivation HKDF ;

— **Échange de clés via key_share** : le serveur inclut sa contribution ECDHE dès le Server Hello, permettant de dériver les clés immédiatement ;

— **Absence de certificat en clair** : en TLS 1.3, le Certificate, CertificateVerify et Finished sont chiffrés et regroupés dans le paquet 18.

6.3.4 d) Le serveur a-t-il envoyé un certificat ?

Oui. Le serveur a bien envoyé un certificat, transmis de façon chiffrée dans le paquet 18. Sans le fichier de clés, ce paquet apparaît comme `Application Data` opaque. Après chargement du fichier `sslkeylog.txt`, Wireshark décode ce paquet et affiche `Handshake Protocol: Certificate` avec une longueur de 3538 octets.

6.3.5 e) Le certificat est-il valide ?

L'analyse du certificat après décryptage révèle les informations suivantes :

Champ	**Valeur**
Subject (CN)	`*.google.com`
Issuer	`GTS CA 1O1` (Google Trust Services)
Organisation	Google LLC, Mountain View, California
Valide du	2020-12-15 14 :42 :53 UTC
Valide jusqu'au	2021-03-09 14 :42 :52 UTC
SAN	`*.google.com`, `*.android.com`, `*.google-analytics.com`, `*.g.co`...

Table 3 – Informations du certificat serveur extrait de la capture

Le certificat était **valide au moment de la capture** (décembre 2020 – mars 2021), mais il a depuis expiré. Le domaine `youtube.com` est couvert via les SAN associés aux domaines Google. L'autorité émettrice `GTS CA 1O1` est une CA intermédiaire de Google Trust Services, dont la chaîne remonte à une Root CA pré-installée dans les navigateurs.

6.3.6 f) Description de l'échange client-serveur

L'établissement de la connexion sécurisée suit les étapes suivantes :

1. **DNS** (paquets 1–3) : résolution de `youtube.com` → `172.217.6.46` ;
2. **TCP Handshake** (paquets 4, 9, 10) : SYN / SYN-ACK / ACK sur le port 443 ;
3. **Client Hello** (paquet 11) : le client envoie ses capacités TLS avec SNI=`youtube.com` ;
4. **Server Hello + Change Cipher Spec** (paquet 16) : le serveur sélectionne TLS 1.3 et `TLS_AES_128_GCM_SHA256` ;

5. **Encrypted Extensions + Certificate + CertificateVerify + Finished** (paquet 18) : authentification du serveur chiffrée ;
6. **Change Cipher Spec + Finished** client (paquet 20) : le client confirme le handshake ;
7. **Application Data HTTP/2** (paquets 21+) : requêtes et réponses HTTP/2 chiffrées.

La particularité de TLS 1.3 est que les clés de session sont dérivées dès le premier échange grâce au mécanisme ECDHE, permettant un handshake complet en **1-RTT** (un seul aller-retour), contre 2-RTT pour TLS 1.2.

6.3.7 g) Nom de domaine visité

Le nom de domaine visité, visible en clair dans le champ SNI du *Client Hello*, est :

`youtube.com`

6.3.8 h) Le domaine visité utilise-t-il HTTPS ?

Oui. La connexion s'effectue sur le **port 443** et utilise le protocole TLS 1.3. Le trafic applicatif utilise **HTTP/2 sur TLS** (HTTPS), confirmé par la négociation ALPN (`h2`) visible dans le Client Hello.

6.4 Analyse avant/après décryptage

Pour décrypter la capture, on charge le fichier `sslkeylog.txt` dans Wireshark via **Édition → Préférences → Protocoles → TLS → (Pre)-Master-Secret log filename**.

Le fichier `sslkeylog.txt` contient les secrets de session au format NSS Key Log : `CLIENT_HANDSHAKE_TRAFFIC_SECRET`, `SERVER_HANDSHAKE_TRAFFIC_SECRET`, `CLIENT_TRAFFIC_SECRET_0`, `SERVER_TRAFFIC_SECRET_0` et `EXPORTER_SECRET`. Trois sessions TLS distinctes sont présentes (trois *Client Random* différents).

Avant décryptage :

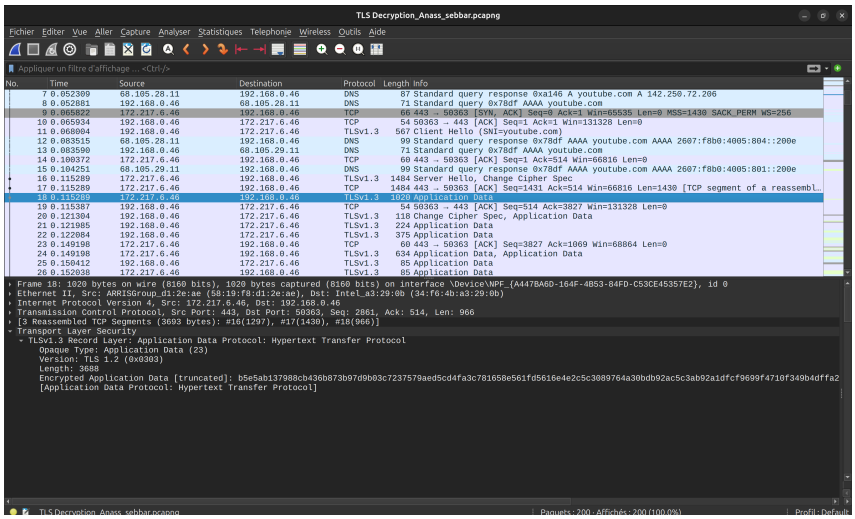

FIGURE 4 – Avant décryptage — Paquet 18 affiché comme `TLSv1.3 Application Data`, contenu chiffré illisible

— Le paquet 18 est affiché comme `TLSv1.3 1020 Application Data` ;

— Le panneau Détails montre `Opaque Type: Application Data (23)` et des octets chiffrés illisibles ;

— Les paquets 21, 22, 24, 25 apparaissent tous comme `TLSv1.3 Application Data` sans information sur le protocole applicatif ;

— On ne peut voir que les métadonnées : adresses IP, ports, taille des paquets, timestamps.

Après décryptage :

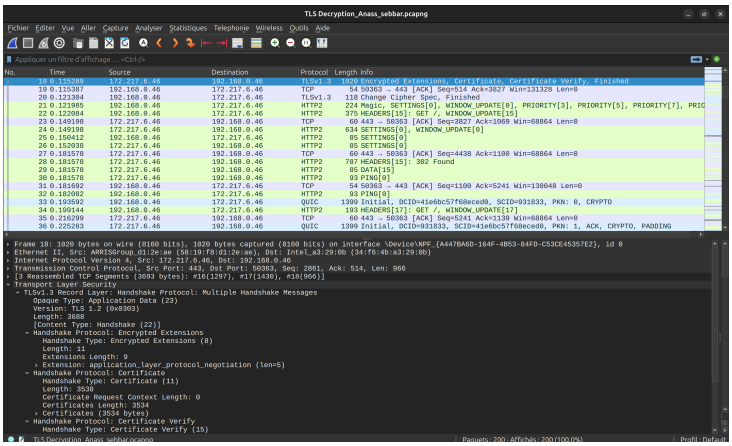

FIGURE 5 – Après décryptage — Paquet 18 révèle le certificat et le Finished ; les paquets suivants passent en HTTP2

— Le paquet 18 affiche désormais : **Encrypted Extensions, Certificate, Certificate Verify, Finished** ;

— Le panneau Détails révèle le certificat (3538 octets) et les messages de fin de handshake ;

— Les paquets suivants (21, 22, 24, 25...) passent du protocole `TLSv1.3` au protocole **HTTP2**.

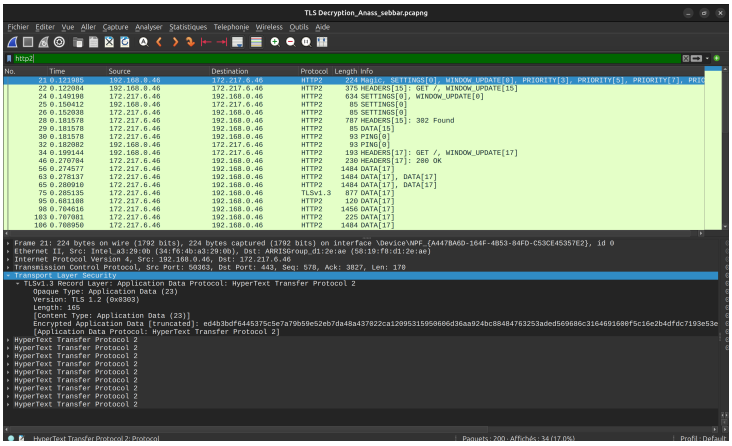

FIGURE 6 – Filtre `http2` après décryptage — Requêtes `GET /`, `302 Found` et `200 OK` visibles en clair

En appliquant le filtre `http2`, on visualise uniquement les paquets déchiffrés. On observe notamment :

- **Paquet 22** : `HEADERS[15]: GET /` — requête vers `youtube.com` ;
- **Paquet 28** : `HEADERS[15]: 302 Found` — redirection vers `https://www.youtube.com/` ;
- **Paquet 34** : `HEADERS[17]: GET /` — deuxième requête vers `www.youtube.com` ;
- **Paquet 46** : `HEADERS[17]: 200 OK` — la page est servie avec succès ;
- Les paquets DATA (56, 63, 65...) contiennent le corps de la réponse HTML/JSON.

Synthèse de la comparaison avant/après :

Élément	**Avant décryptage**	**Après décryptage**
Protocole affiché	TLSv1.3 Application Data	HTTP/2
Paquet 18	Application Data opaque	Certificate + Finished visibles
Contenu des paquets	Octets chiffrés illisibles	Requêtes/réponses HTTP/2
Méthode HTTP	Non visible	GET /
Domaine cible	`youtube.com` via SNI	`youtube.com` + `www.youtube.com`
Code de réponse	Non visible	302 Found, puis 200 OK
User-Agent	Non visible	Firefox 86 sur Windows 10

Table 4 – Comparaison avant/après décryptage TLS

Conclusion

Ce TP a permis d'approfondir la compréhension des protocoles de sécurité réseau à travers l'analyse pratique avec Wireshark. Les points clés retenus sont les suivants :

- Wireshark est un outil puissant pour l'analyse du trafic réseau, permettant d'isoler précisément les paquets d'intérêt grâce aux filtres d'affichage ;
- Le protocole **HTTP** est fondamentalement non sécurisé : un attaquant sur le LAN peut voir, modifier et rejouer tout le trafic ;
- Le protocole **HTTPS/TLS** assure la confidentialité et l'intégrité du contenu, mais le **nom de domaine reste visible** dans le champ SNI du Client Hello — comme observé avec `youtube.com` dans notre capture ;
- Les sites de phishing peuvent utiliser des certificats TLS **valides** : la présence du cadenas HTTPS ne garantit pas la légitimité d'un site ;
- L'analyse du handshake TLS révèle de nombreuses informations : version négociée (**TLS 1.3**), cipher suite choisie (`TLS_AES_128_GCM_SHA256`), identité du serveur (certificat Google signé par `GTS CA 1O1`) ;
- Avec le fichier `sslkeylog.txt`, Wireshark peut déchiffrer intégralement le trafic TLS et révéler les requêtes HTTP/2 en clair — une illustration concrète de l'importance de la protection des clés de session.